The Second Brain: An AI Research System You Can Audit

How it is built, how it remembers, how it reasons, and how to use it on any project

Prepared by Isa Haskologlu, PhD · © 2026 · CC BY 4.0

2026-09-29

Contents

Read This First 1

1. The Problem It Solves 2

2. The System at a Glance 2

3. The Components, One Zoom Level Down 4
- 3.1 Obsidian: the knowledge base . . . 4
- 3.2 The rulebook . . . 6
- 3.3 The source pipeline: from PDF to trustworthy text . . . 6
- 3.4 Memory engineering: what is remembered, where, and why . . . 7
- 3.5 Graph engineering: two kinds of graph . . . 7
- 3.6 Integrations and what each one adds . . . 11
- 3.7 Four building blocks, one system . . . 17

4. Worked Example: Follow One Source Through the System 20

5. One Core, Many Projects 21

6. Before and After 23

7. Honest Limits 24

8. Glossary 24

Appendix A. Build Your Own: A Tell–Show–Do Setup Guide 25

Appendix B. Sources and Provenance for This Guide 31

License and How to Cite 32

Read This First

The answer in one sentence. The Second Brain is an AI research assistant (Claude Code) working inside a plain-text knowledge base (an Obsidian vault) under written rules. Every claim it keeps can be traced to a source page, every decision is recorded, and every output can be audited.

Three things to remember:

1. **It remembers on purpose.** Knowledge, project status, decisions and preferences each go into a different, inspectable place. Nothing important lives only inside the AI's "memory".
2. **It works like a well-run research team, not a chatbot.** Work is split into steps with explicit dependencies. Independent evidence is gathered in parallel, and a fresh reviewer checks results before anything is written.
3. **It is general-purpose.** The same core ran a terrorism data analysis and a (hypothetical) investment review. The core stays the same; only the project folder changes.

What it is not. It does not make AI output true. It makes AI output *checkable*. Most notes in the vault are still marked "needs verification", and that label is shown, not hidden.

---

1. The Problem It Solves

Situation. Researchers, analysts and executives already use AI assistants to read, summarize and draft.

Complication. A general chatbot has four structural weaknesses for serious work:

Weakness	What it looks like in practice
It forgets	Every new chat starts from zero. Context, decisions and preferences are lost.
It invents	Citations, page numbers and quotations can be fabricated and still sound plausible.
It cannot show its work	There is no trail from a statement back to the page it came from.
It does not scale	Long projects exceed what fits in one conversation.

Question. How do you get AI speed without giving up the standards of research: provenance, verification and reproducibility?

Answer. Put the AI inside a system that enforces those standards: a structured knowledge base, written operating rules, an explicit memory design and a workflow graph. That system is the Second Brain.

> **Analogy for non-technical readers.** Think of a well-run research lab: the **library** holds original sources untouched (`01–Library/`); the **lab notebook** records one finding per page with its citation (`02–Wiki/`); each **project binder** holds status and decisions (`03–Projects/`); the **lab manual** says how work must be done (`CLAUDE.md`, `RESEARCH_PROTOCOL.md`); the **peer reviewer** checks work before it is filed (fresh-context verification); and a **calibrated second opinion** (Jev) sorts repeated, narrow judgments into clear and uncertain cases. It is an aid to judgment, never a source of evidence. Claude is the tireless research assistant who works in this lab and follows the manual.

---

2. The System at a Glance

The system has five layers. Information flows from the top down; rules apply to every layer.

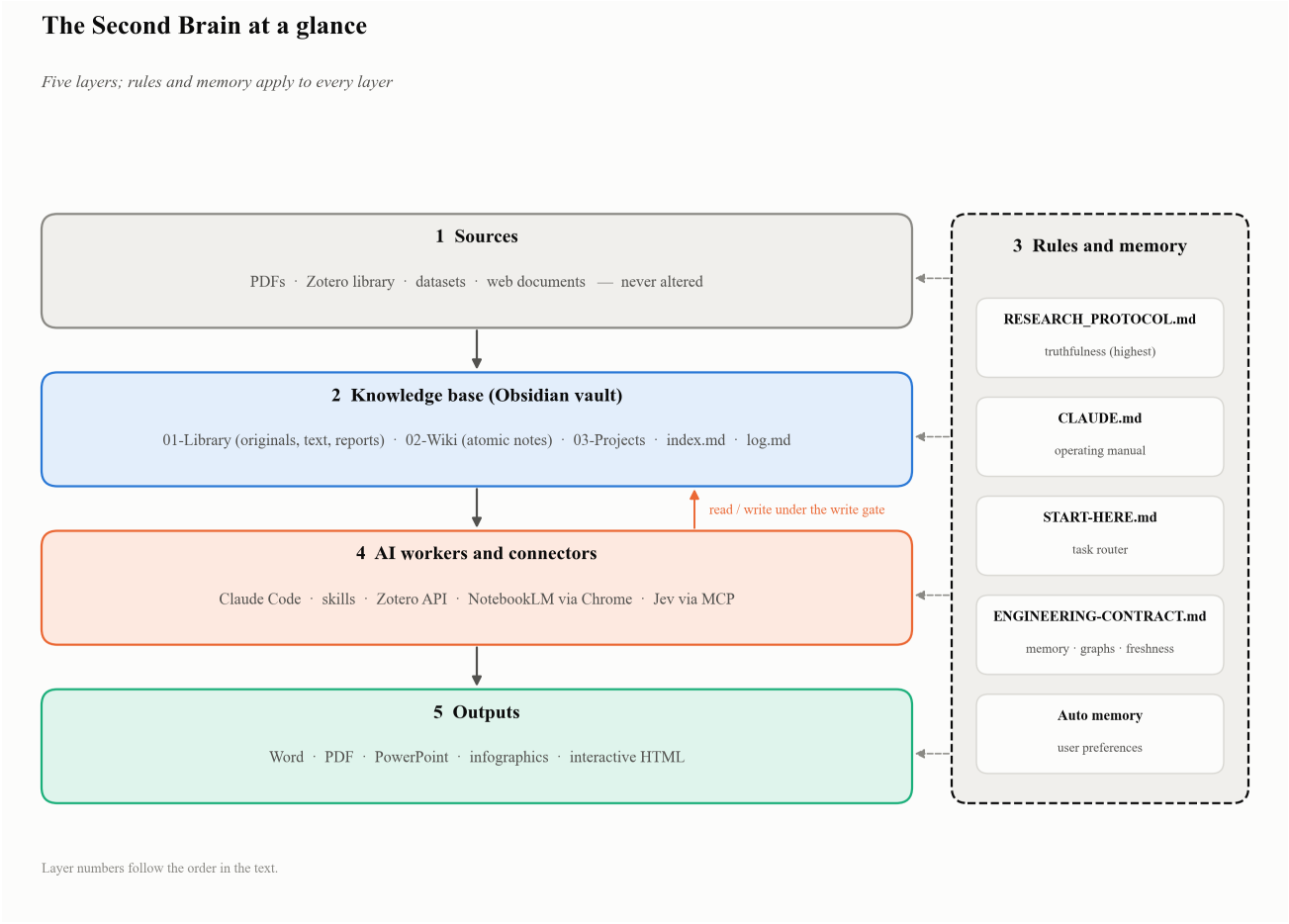


Figure 1: System architecture

Layer	Components	Role
1. Sources	PDFs, Zotero library, datasets, web documents	Raw material, never altered
2. Knowledge base	Obsidian vault: `01-Library/`, `02-Wiki/`, `03-Projects/`, `index.md`, `log.md`	Durable, linked, plain-text memory
3. Rules and memory	`CLAUDE.md`, `RESEARCH_PROTOCOL.md`, `00-System/START-HERE.md`, `00-System/ENGINEERING-CONTRACT.md`, Claude auto-memory	How work is done and what is remembered
4. AI workers and connectors	Claude Code, skills, Zotero API, NotebookLM (via Chrome), Jev (via MCP)	Do the reading, checking and building
5. Outputs	Word, PDF, PowerPoint, infographics, interactive HTML	Deliverables for people

What the vault holds:

Part	What it holds
Atomic wiki notes	One idea per note, each with its sources; new notes stay marked needs-verification until a person checks them
Links between notes	Explained relationships that connect ideas across notes
Original source files	PDFs and books, kept exactly as they arrived
Extracted text	Searchable text with page markers, so notes can cite pages
Activity log	A dated record of each work session
Project workspaces	One folder per project

---

3. The Components, One Zoom Level Down

3.1 Obsidian: the knowledge base

Obsidian is a note-taking application that works on a folder of plain Markdown files. The vault is that folder. Three properties make it suitable:

- **Plain text.** Every note is a readable `.md` file. There is no lock-in: Git, any editor and any AI tool can read it.
- **Wikilinks and graph view.** `[[note-name]]` links turn notes into a navigable network, and Obsidian draws it as a graph.
- **Metadata.** Each wiki note starts with a YAML header: title, dates, language, tags, page type, **verification status** and **sources**.

Folder map.

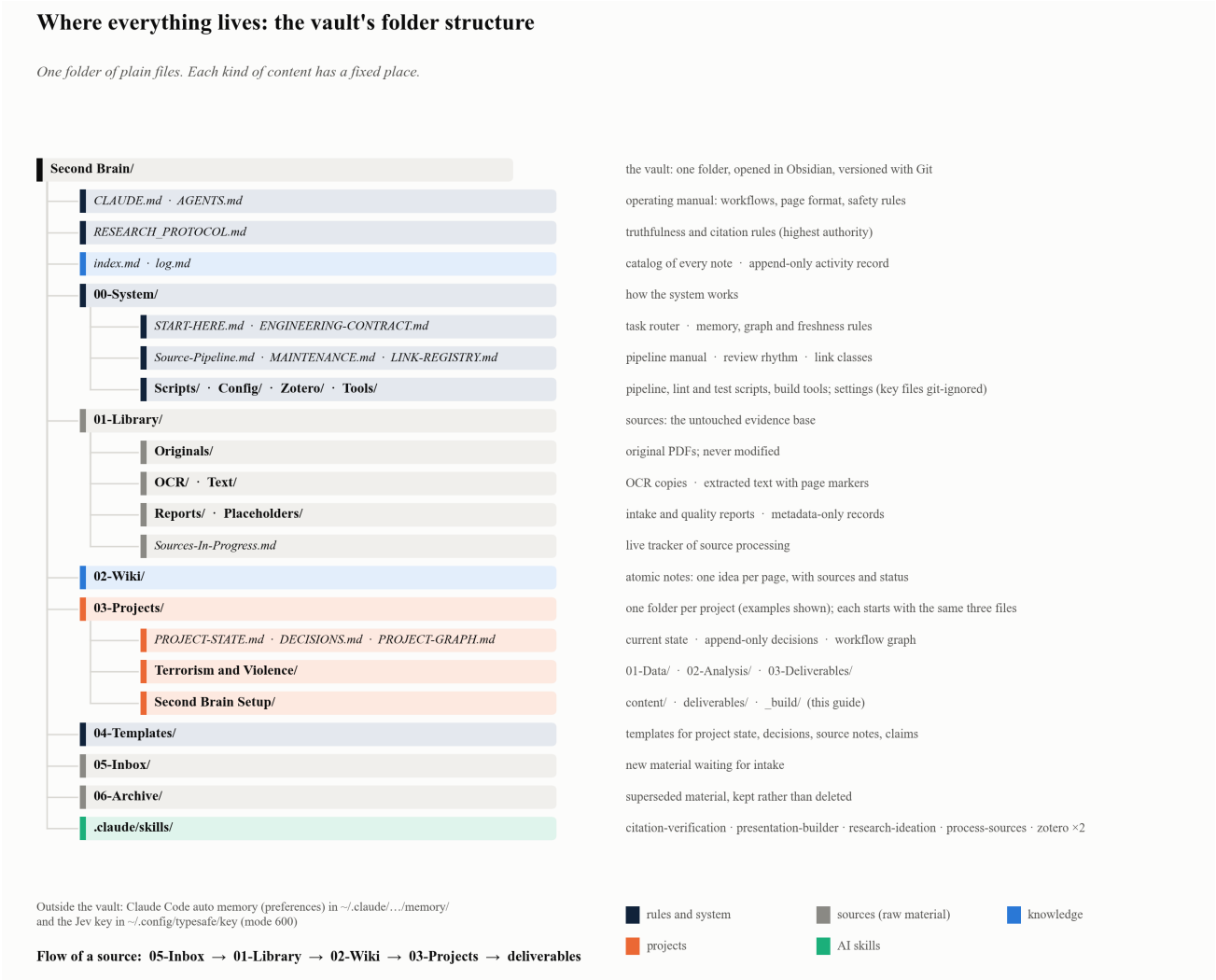


Figure 2: The vault's folder structure: where each kind of file lives

Folder	Holds	Rule
`00-System/`	Operating manuals, scripts, configuration	Changed deliberately
`01-Library/`	Originals, OCR'd copies, extracted text, quality reports	**Originals are never modified**
`02-Wiki/`	Atomic notes: one idea per page	Neutral: never bent to fit a project thesis
`03-Projects/`	One folder per project: state, decisions, graph, deliverables	Project-specific rules may add, never weaken
`04-Templates/`	Reusable note templates	—
`05-Inbox/`	New material before intake	Temporary
`index.md` / `log.md`	Catalog of all notes / chronological activity record	Log is append-only

3.2 The rulebook

Four documents govern every session, in a fixed order of authority:

1. **`RESEARCH_PROTOCOL.md`**: highest authority on truthfulness. No invented citations, quotations or metadata. Source claims are kept separate from interpretation. Uncertainty and disagreement are stated openly.
2. **`CLAUDE.md`**: the operating manual. Defines workflows (`ingest`, `query vault`, `save`, `lint wiki`, `weekly review`), page structure, file naming and change-safety rules.
3. **`00-System/START-HERE.md`**: the router. It tells the AI which few files to load for a given task, and in which order.
4. **`00-System/ENGINEERING-CONTRACT.md`**: memory types, the write gate, freshness, context budgets, workflow graphs and link rules.

Why a written rulebook matters. Behaviour is set by files that anyone can read, change and version-control, not by hidden prompts. Every session loads the same rules.

3.3 The source pipeline: from PDF to trustworthy text

Stage	What happens	Tool
0 Acquisition	Source placed in `05-Inbox/` or imported from Zotero	Manual / `zotero_import.py`
1 Intake review	Is there a text layer? Which language?	`inspect-pdf.py`
2 Format preparation	OCR if scanned; text extraction; **page markers** added	`process-source.py`, `add-page-markers.py`
3 Quality review	Garbled-character detection, page correspondence → `quality_status`	`validate-text.py`
4 Neutral ingestion	Atomic wiki notes with page-level citations	`ingest` workflow
5 Verification	A person checks claims and quotations against the original	`citation-verification` skill
6 Project use	A project applies the knowledge under its own stricter rules	Project folder

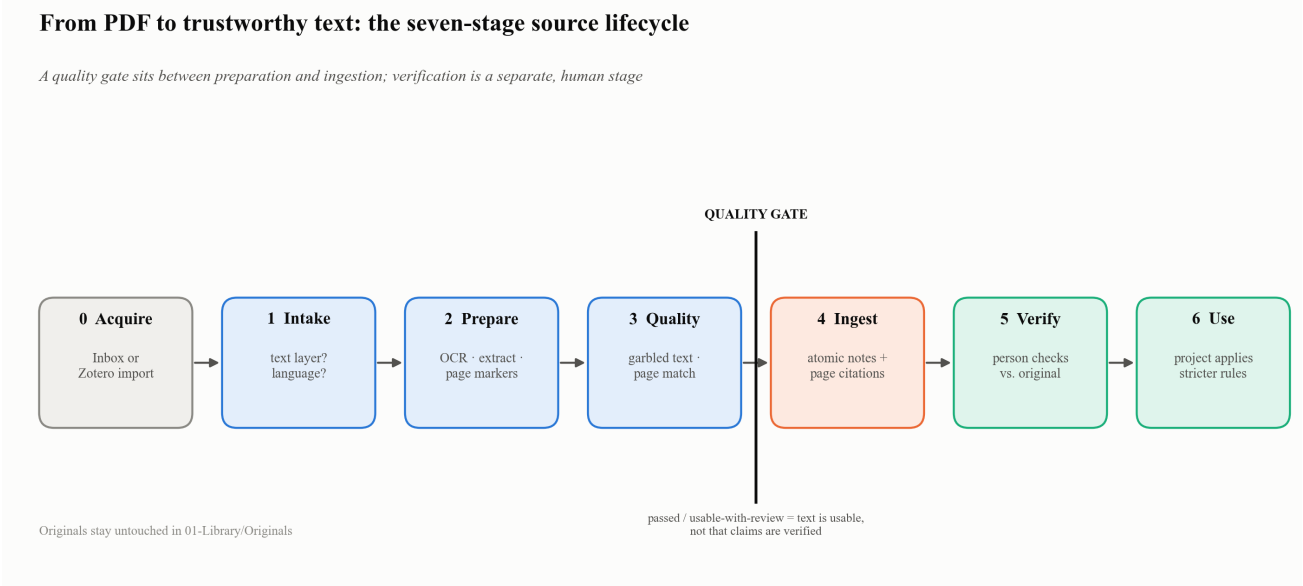


Figure 3: Seven-stage source lifecycle

Quality gate. A source must reach `passed` or `usable-with-review` before ingestion. Passing means the *text is technically usable*. It does **not** mean the claims are verified; that happens only at Stage 5.

3.4 Memory engineering: what is remembered, where, and why

A chatbot has one kind of memory: the conversation. The Second Brain splits memory by *function*, so each kind can be found, checked and corrected.

Memory layer	What belongs there	What does **not**	Where it lives
Project instructions	Durable workflow, scope, permissions, citation and verification rules	Substantive facts, passwords, speculation	`CLAUDE.md`, project system files
Verified source notes	Bibliographic data, exact support, pages, methods, limits, verification status	Model guesses or unsourced summaries presented as fact	`02-Wiki/`, `01-Library/`
Evidence / claim graph	Explicit support, contradiction, method and scope relationships	Decorative keyword links, hidden reasoning	Wikilinks, `index.md`, hubs
Project state and decisions	Current phase, blockers, next actions; append-only decision record	Chat summaries treated as evidence	`PROJECT-STATE.md`, `DECISIONS.md`
Task prompt / session	One-time objective, output format, acceptance tests	Rules that must survive future sessions	The conversation
Auto memory	Audited, non-sensitive, reusable preferences (e.g., "all outputs in English, Times New Roman")	Evidence, confidential data, stale conclusions	Claude Code memory folder, outside the vault

(The layer table is adapted from the author's graduate course materials on research systems.)

Four rules make memory trustworthy:

- **Write gate.** Before anything is saved: was the write authorized? Which memory type is it? Does an equivalent record already exist? Is provenance preserved? Is the change logged?

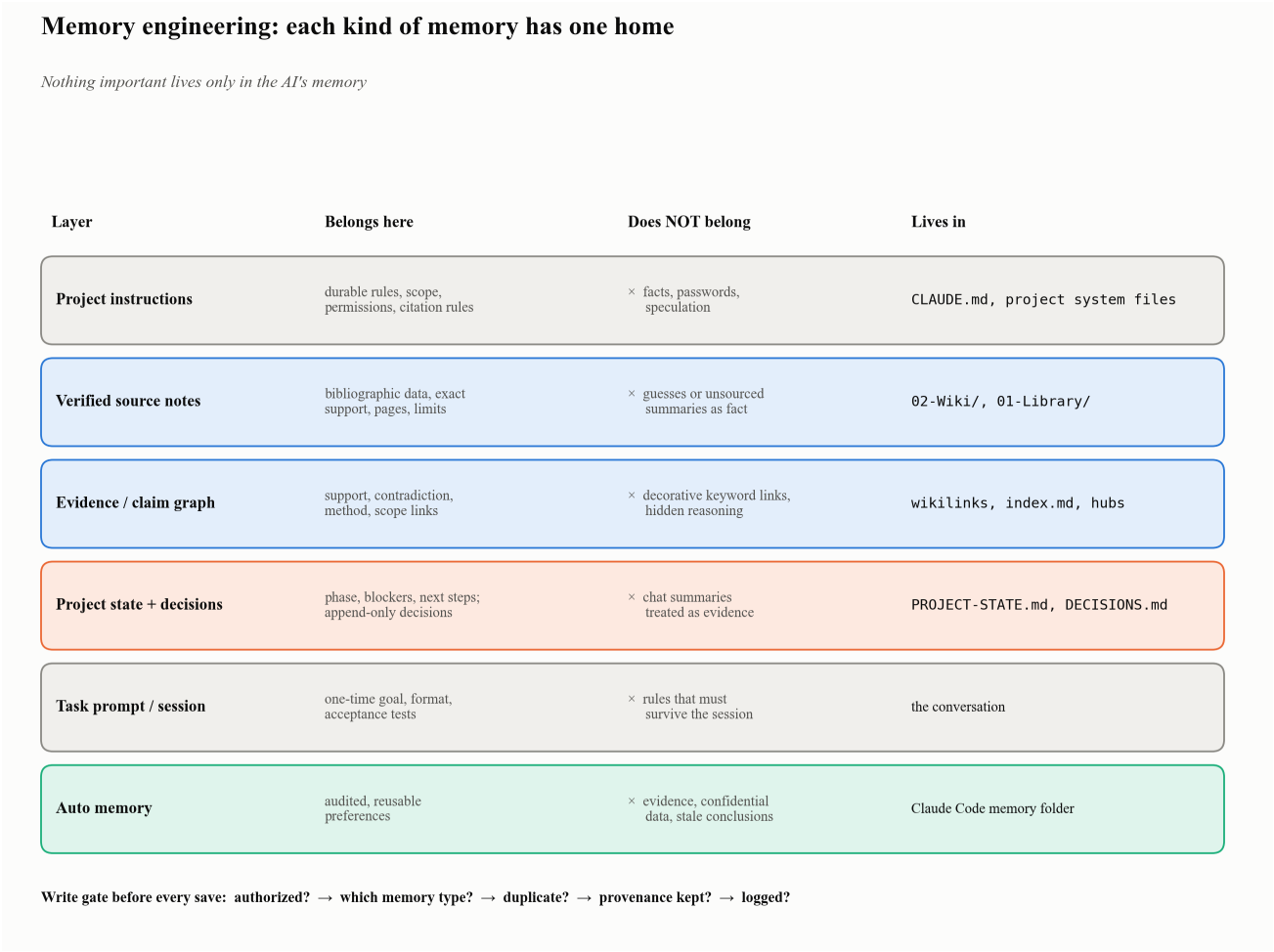


Figure 4: Memory architecture

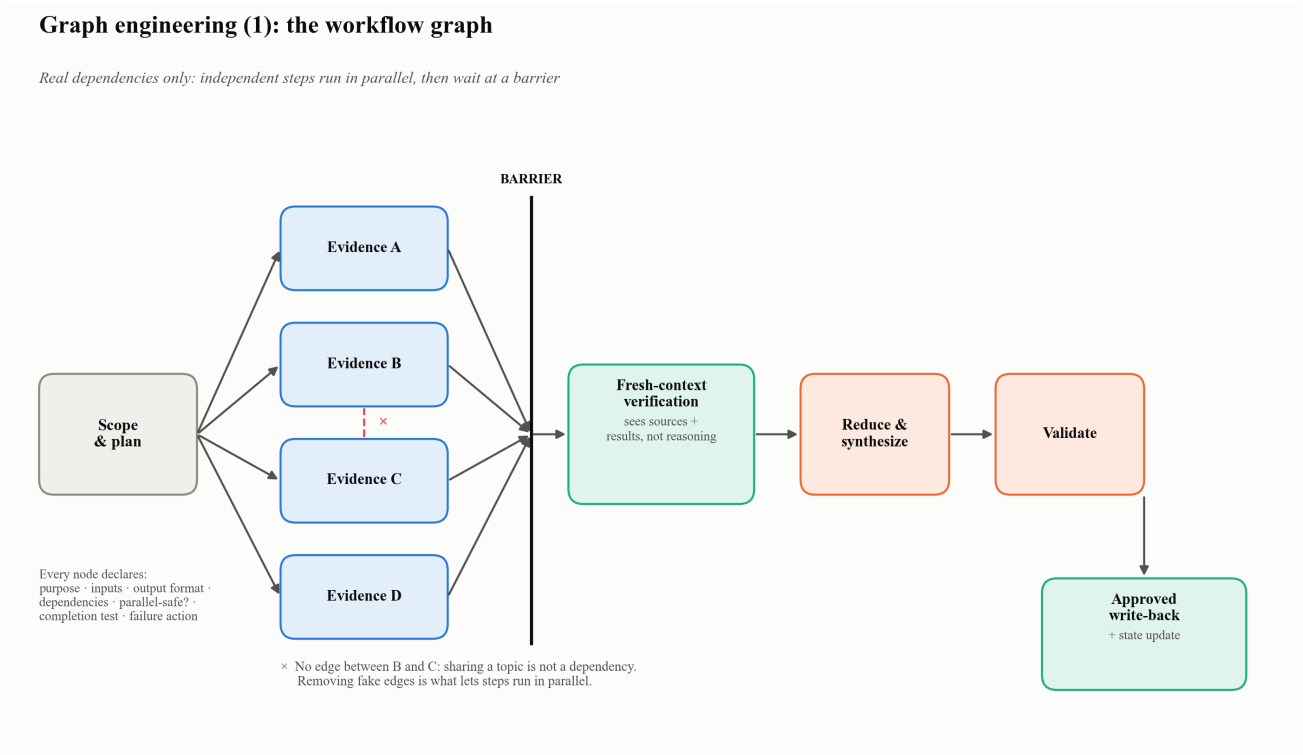


Figure 5: Workflow graph with fan-out, barrier and fresh-context verification

```
→ independent evidence nodes (fan-out, in parallel)
→ evidence-complete barrier
→ fresh-context verification
→ reduce and synthesize
→ schema and quality validation
→ approved write-back and state update
```

Each node declares its purpose, inputs, output format, dependencies, whether it can run in parallel, its completion test and what happens if it fails.

- **Real edges only.** Draw an edge only if the next step needs the previous step's *data* or *decision*. "They are on the same topic" is not an edge. Removing these fake edges is what allows parallel work.
- **Fresh-context verification.** The checker receives the scope, the sources and the normalized results, but *not* the worker's reasoning. It re-tests claims against sources, as an independent reviewer would.
- **Per-node context.** Each step sees only what it needs. Small contexts give better focus and fewer errors.

Example (Terrorism and Violence project). Data acquisition branched into independent analyses: geographic hot spots, group operations, designation cross-checks. These waited at a barrier, then fed a forecasting step and a Europe/US risk step. A fresh-context audit re-computed the published numbers before the report, workbook, decks and interactive page were finalized.

(b) The knowledge graph: how knowledge connects

Notes link to each other with typed, explained relationships: claim → supporting source → method → limitation → contradiction. The rules:

- A link must explain a relationship, not just a shared keyword.

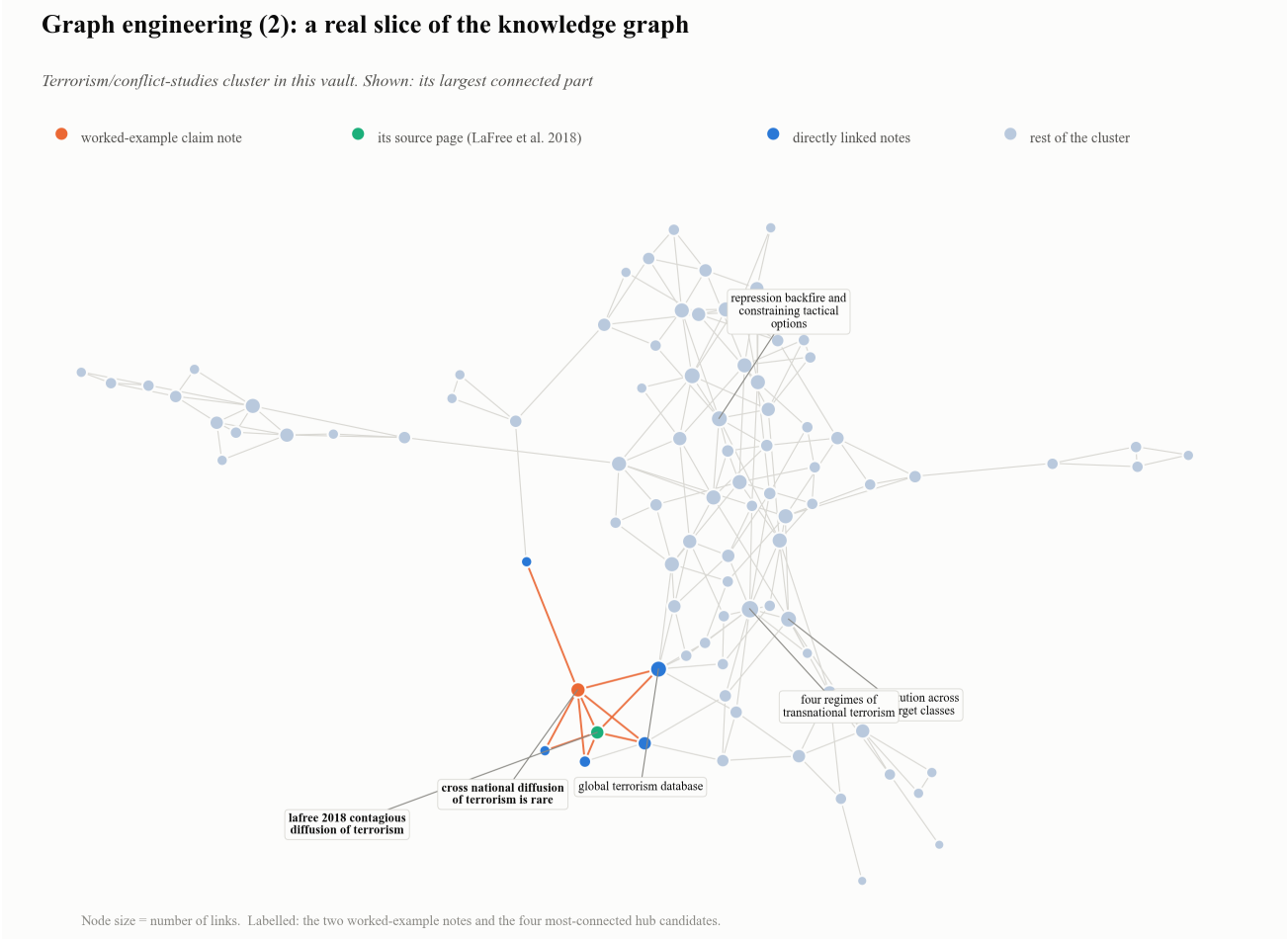


Figure 6: A real slice of the vault's knowledge graph: the terrorism/conflict-studies cluster, with the worked-example notes highlighted

- Missing link targets are classified as **resolved**, **planned** (a deliberate future note), **broken**, **renamed** or **external-to-wiki**. They are not "fixed" by creating empty pages.
- Notes that connect three or more substantial pages are flagged as **hub** candidates.
- A read-only linter (`lint_vault.py`) reports orphans, broken links and missing metadata without changing anything.

Why it matters. Retrieval is **index-first and bounded**: the AI searches `index.md`, loads at most five notes per batch, and follows links only to fill a named gap. It never reads the whole vault. This keeps answers grounded and the context small.

3.6 Integrations and what each one adds

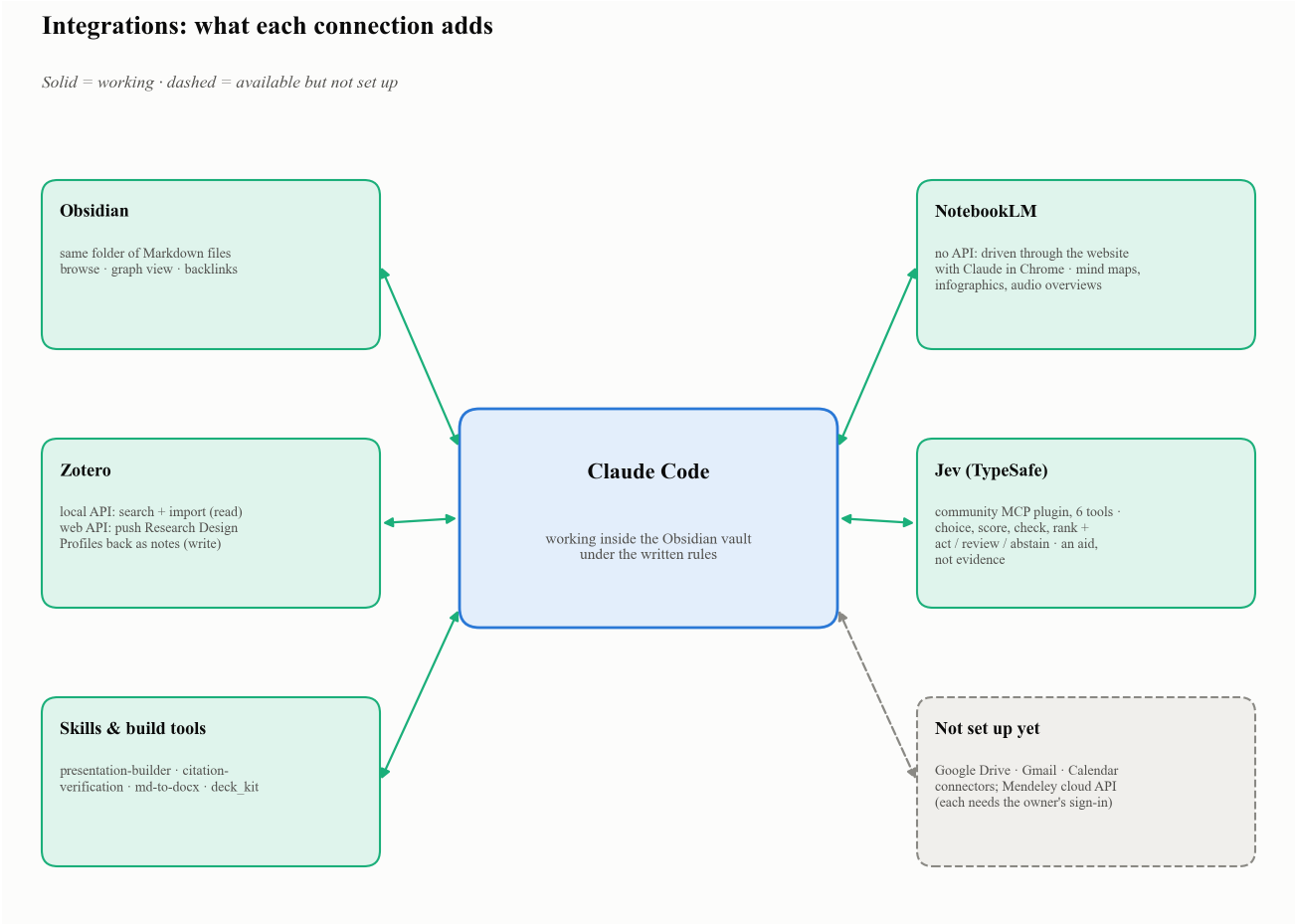


Figure 7: Integration map

Integration	How it connects	What it adds	Observed limits
Obsidian	Same folder; Claude reads and writes Markdown	Human-friendly browsing, graph view, search, backlinks	Obsidian itself has no AI in this setup; it is the interface

Integration	How it connects	What it adds	Observed limits
Zotero	Local API (read-only, on the same machine) for search and import; Web API with a write key for pushing notes back	Bibliographic metadata with provenance; PDFs flow into the pipeline. After ingestion, a **Research Design Profile** (question, argument, theory, variables, method, data, findings, verbatim abstract) is written back to Zotero as a child note	Local API cannot write; the write key is stored in a git-ignored file
NotebookLM	No public API. Claude operates the NotebookLM website through **Claude in Chrome** (browser automation)	Formats Claude does not produce itself: mind maps, NotebookLM infographics, audio overviews; cross-source questions over a notebook	Usage quotas can halt a batch; outputs go stale when upstream facts are corrected and must be regenerated
Jev (TypeSafe)	Community-maintained Claude Code plugin (`jev@jev-mcp`) that bundles an MCP server with **six decision tools** and an agent skill; authenticated by a TypeSafe API key stored in a protected file	Bounded, **calibrated** judgments (choice, score, check, ranking, triage) with probabilities, a confidence value and an **act / review / abstain** recommendation	A decision aid only: it does not make a source true, verify a citation, or replace domain judgment. Not an official TypeSafe or Anthropic product
Skills	Packaged instructions Claude loads on demand	Repeatable workflows: `presentation-builder`, `citation-verification`, `research-ideation`, `zotero-sources`, `process-sources`	Only as good as the written procedure
Build tools	Shared scripts in `00-System/Scripts/`	One-command Word (`md-to-docx.sh`), decks (`deck_kit.py`), forecasting bake-off (`forecast_bakeoff.py`)	Standards (English, Times New Roman) are enforced in code
Available, not set up	Google Drive, Gmail and Calendar connectors; Mendeley cloud API	Would add document and schedule context; a second reference manager	Each needs the owner's own sign-in (OAuth); not yet authorized

Jev: bounded judgments that show their uncertainty

Claude gives Jev a defined task, the allowed answers and the relevant text. Jev returns a **typed result** instead of a free-form essay:



Figure 8: Conceptual view of a calibrated Jev decision: clear cases proceed, borderline cases go to a person, unsupported cases are set aside. Illustration from the Jev installation deck

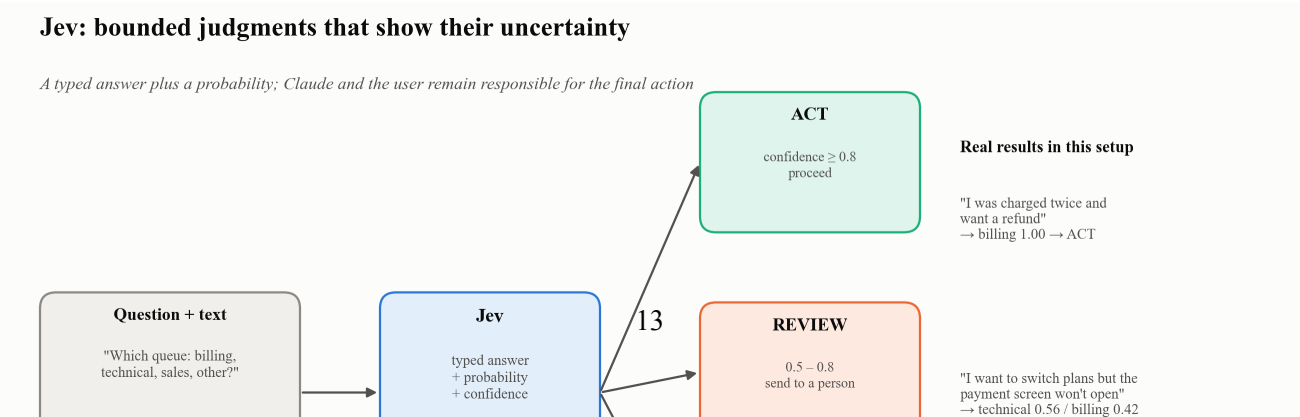
- a choice, score, check or ranking;
- probabilities and a confidence value;
- a recommendation to **act**, **review** or **abstain**.

Claude and the user remain responsible for the final action.

Why it matters. Jev improves *repeated* decisions by separating clear cases from uncertain ones. The value appears when one semantic rule must be applied across many items, and uncertainty should change the next step. The results are more consistent labels across a batch, faster prioritisation of material to inspect, and a visible review path for borderline cases. Typical uses are research screening, dataset labelling and course-material triage.

Division of labour (from the companion installation guide):

Part	Best use
Claude	Reasoning, writing, planning, interpretation
Jev	Choice, yes/no probability, scoring, ranking, triage
Ordinary code	Counting, dates, formulas, thresholds, file operations



anything.

Real results from this setup:

- A clear customer message ("My card was charged twice and I want a refund") → *billing*, probability 1.00, confidence 1.00 → **act** (255 ms).
- An ambiguous one ("I want to switch plans but the payment screen won't open") → *technical* 0.56 vs. *billing* 0.42, confidence 0.42 → **abstain**. The system declined to guess. That is the behaviour you want.

  What Jev is not. "Jev is a decision aid. It does not make a source true, verify a citation, or replace domain judgment." Its output is never presented as research evidence, factual verification, citation verification or source validation. It is not used for arithmetic or date comparison. (Source: the audited Jev installation deck.)

Jev and Claude: fast judgments and deep reasoning

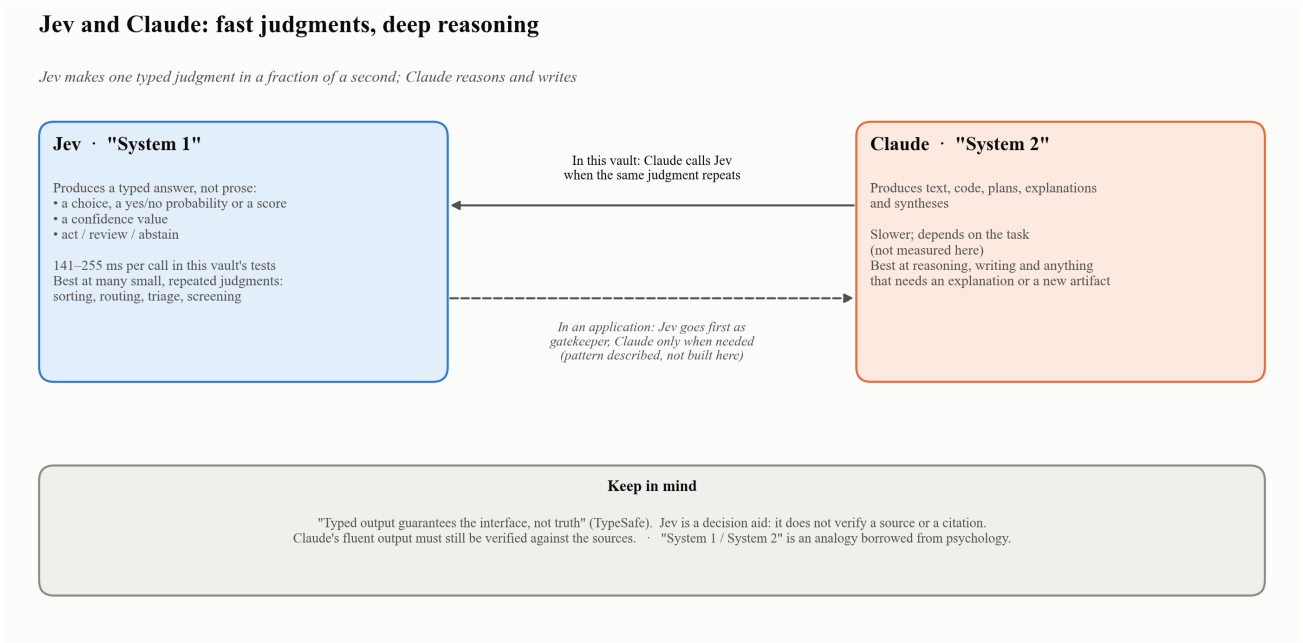


Figure 10: Jev as "System 1", Claude as "System 2"

TypeSafe describes Jev as its first **"System One" model**: it "understands natural language and returns typed answers and probabilities rather than generating text or reasoning explanations" (TypeSafe skill documentation). The name borrows a familiar distinction from psychology (external analogy): fast, instinctive **System 1** judgments versus slow, deliberate **System 2** reasoning.

	Jev (System 1)	Claude (System 2)
Produces	Typed answers: a choice, a yes/no probability, a score, with a confidence value and an act / review / abstain recommendation. No prose.	Text, code, plans, explanations, syntheses
Speed	141–255 ms per call in this vault's tests	Longer; depends on the task (not measured here)

	Jev (System 1)	Claude (System 2)
Best at	Many small, repeated judgments: sorting, routing, triage, screening	Reasoning, writing, anything that needs an explanation or a new artifact
Keep in mind	"Typed output guarantees the interface, not truth" (TypeSafe)	Fluent output must still be verified against sources

Two ways to combine them.

1. **In this vault, Claude calls Jev.** Claude runs the workflow and calls Jev through MCP whenever the same bounded judgment repeats, for example classifying many notes or routing ambiguous cases to review.
2. **In an application, Jev can go first as a gatekeeper.** Code sends every incoming item to Jev ("Is this support email urgent?", "Is this change risky?") and calls Claude only when deep reasoning or new text or code is needed. TypeSafe's own summary of the pattern: "Code owns the workflow; the model supplies programmable common sense." *This pattern is described here for completeness; it has not been built in this vault.*

Names you may see. In TypeSafe's SDK the judgment types are called **Choice** (pick one option), **Noul** (probability that a condition holds) and **Score** (position on an ordered scale). In Claude Code, the same judgments appear as MCP tools: `jev_classify`, `jev_check`, `jev_score`, plus `jev_ask`, `jev_triage` and `jev_models`.

Pricing is not covered here; check TypeSafe's current documentation.

Jev and graph engineering: different things that work together

They are easy to confuse because both appear in "decision" diagrams, but they are different kinds of thing:

	Jev	Graph engineering
What it is	A model: one component	A design method: how the steps of a task are organised
Its job	Make one fast, typed judgment (choice, yes/no probability, score)	Define the steps (nodes), the real dependencies between them (edges), barriers and verification
Analogy	The clerk who stamps each file "approved" or "rejected" in an instant	The organisation chart and workflow manual that say which department does what
In this vault	MCP tools `jev_classify`, `jev_check`, `jev_score`, …	`00-System/ENGINEERING-CONTRACT.md` §6 and each project's `PROJECT-GRAPH.md`
Outside this vault	TypeSafe SDK primitives Choice, Noul, Score	Agent-graph frameworks such as LangGraph, CrewAI or LlamaIndex (external examples; not used here)

How they fit together. A graph has *decision* or *routing* nodes, points where the workflow must choose which way to go. Traditionally that choice is made by a large model such as Claude, which is slower and costlier than needed for a narrow decision. Jev fits inside exactly those nodes: it picks the branch in a fraction of a second, and the graph's edges take the work where it needs to go.

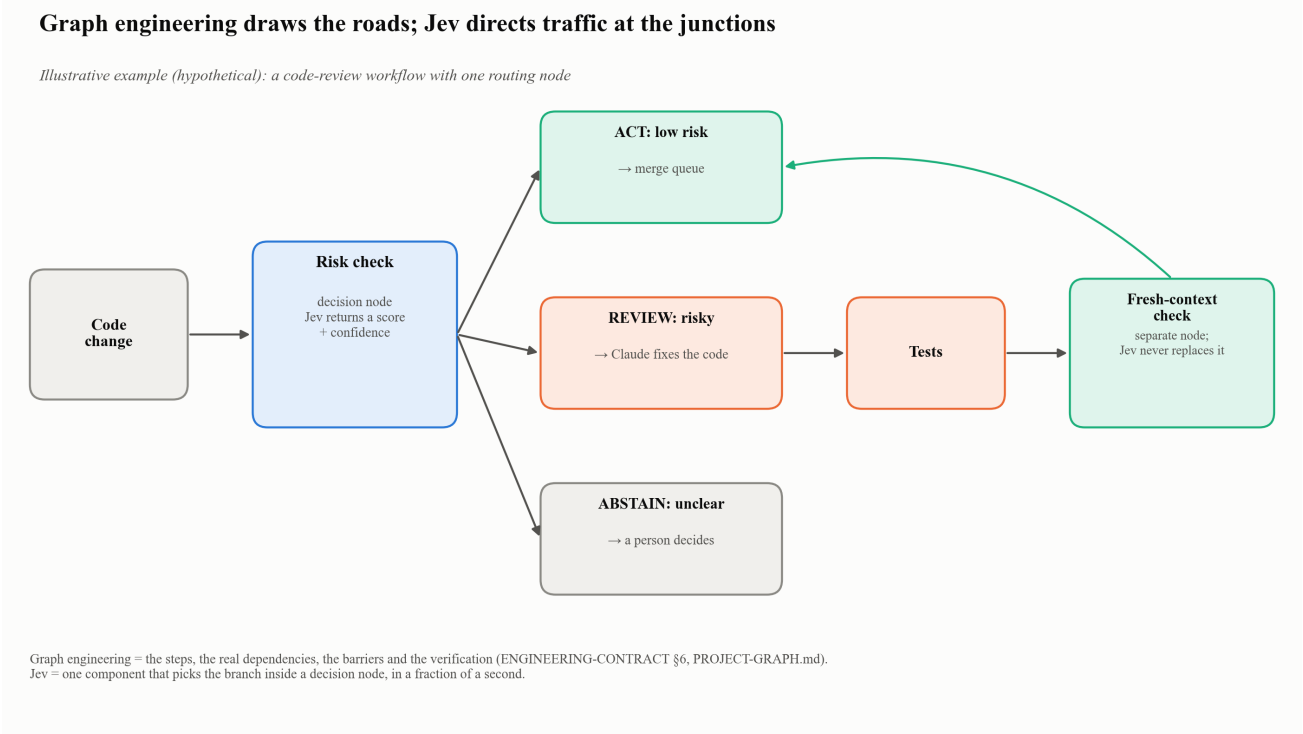


Figure 11: Graph engineering draws the roads; Jev directs traffic at the junctions

Illustrative example (hypothetical). A code-review workflow:

1. *Graph engineering* defines the path: code change → **risk check** → (if risky) Claude fixes the code → tests → fresh-context check.
2. *Jev* sits inside the risk-check node and returns a risk score with a confidence value.
3. The graph routes on that answer: **act** (low risk) → merge queue; **review** (risky) → Claude, working as System 2, fixes the code; **abstain** (unclear) → a person decides.

One rule does not change: Jev picks the branch, but it never replaces the verification node. The graph is the skeleton and the roads; Jev is the fastest traffic officer at the junctions.

Installing Jev: three components, eight steps

Component	What it is
Claude Code	The terminal assistant where you type commands and prompts
Jev MCP plugin	Six decision tools for classification, scoring, checking and triage; the plugin also installs its agent skill
TypeSafe API	The hosted service and API key that let Jev make calibrated judgments

Commands go into **two different interfaces**. Slash commands (`/plugin`, `/mcp`, `/reload-plugins`) are typed *inside Claude Code*. Shell commands (`claude plugin list`, the key file) are typed in the *normal Terminal*. When a command fails, check first that it went into the right one.

#	Step	Where	Check
1	Create a TypeSafe API key (console.typesafe.ai → Settings → Keys)	Browser	Key copied once; never pasted into chat
2		Inside Claude Code	"Successfully added

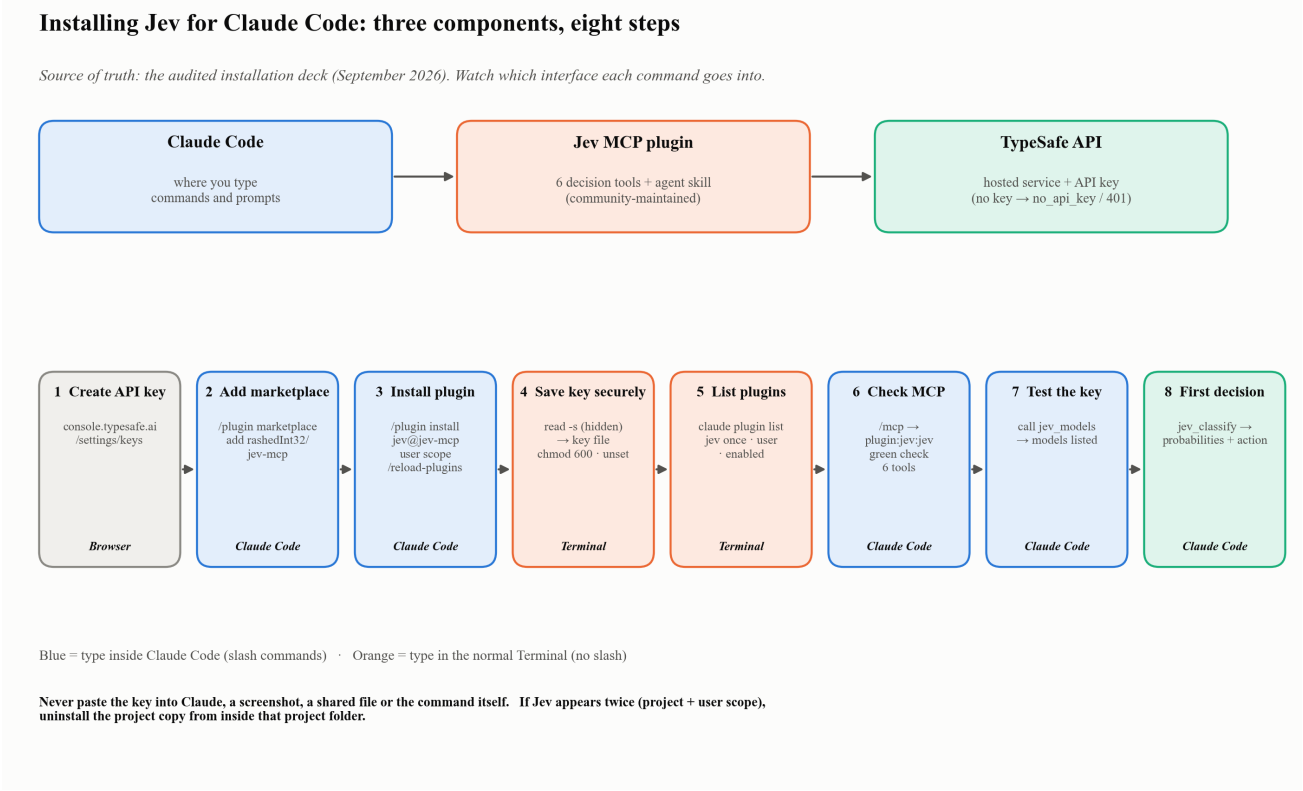


Figure 12: Installing Jev for Claude Code

The full command-by-command version, with troubleshooting, is in Appendix A, Step 6.

3.7 Four building blocks, one system

A modern AI assistant is no longer “one prompt, one answer”. It plans steps, remembers, decides and checks its own work. Four ideas describe how, and they are often confused:

	Memory engineering	Graph engineering	Jev (TypeSafe)	Loop engineering
What it is	A method: decide what is remembered, where, and how it is retrieved	A method: map the task as steps (nodes) and real dependencies (edges)	A model: a fast component that makes one typed judgment	A method: produce → check → fix → re-check
Its job	Prevent “amnesia”; keep long projects consistent across sessions	Make multi-step work manageable, parallel and traceable	Make quick routing and triage decisions without a large model	Catch and fix errors before a person sees the output
Workplace analogy	The company archive and client files	The organisation’s process chart: who does what, in which order, with which approvals	The receptionist who reads each document and instantly sends it to the right desk	The careful employee who re-checks a report before handing it in

	Memory engineering	Graph engineering	Jev (TypeSafe)	Loop engineering
In this vault	Plain files: `PROJECT-STATE.md`, `DECISIONS.md`, `02-Wiki/`, `log.md`, auto memory; index-first retrieval; write gate. **No vector database.**	`ENGINEERING-CONTRACT.md` §6 (node contract, real edges, barriers); each project's `PROJECT-GRAPH.md`	MCP tools `jev_classify`, `jev_check`, `jev_score`, `jev_triage`, `jev_ask`, `jev_models`	Fresh-context verification; on failure, return to the failing node, retry only when safe, then escalate to a person (ENGINEERING-CONTRACT §6); weekly lint and review (MAINTENANCE.md); QA-and-fix cycles on every deliverable
Outside this vault (examples)	Vector databases (e.g., Milvus/Zilliz), Mem0, LangChain memory, Redis	LangGraph, CrewAI, LlamaIndex Workflows	TypeSafe SDK primitives Choice, Noul, Score	ReAct-style agent loops; critic/reflection loops; custom while-loops

The "outside" column lists commonly cited external tools as examples only; none is used or evaluated in this vault.

How they work together in this vault

1. **Memory: remember before acting.** Each session starts with the router and the project's current state (`START-HERE.md` → `PROJECT-STATE.md` → `index.md`), so the work starts from what was already decided, not from zero.
2. **Graph: draw the map.** `PROJECT-GRAPH.md` defines the steps and their real dependencies: scope → parallel evidence → barrier → verification → synthesis → approved write.
3. **Jev: route quickly at the decision points.** Where the same narrow judgment repeats (which of 50 files deserve a close read? which queue does this item belong to?), Jev answers in a fraction of a second, with act / review / abstain.
4. **Claude: reason and write.** Claude does the deep work: reading, synthesising, drafting.
5. **Loop: check, fix, re-check, within limits.** A fresh-context verifier re-tests the claims against the sources. If a check fails, the work goes back to the failing node. Retries are allowed only when safe and recorded, and anything still unresolved goes to a person, never "loop until it looks right".
6. **Memory again: write back.** Only verified results pass the write gate into the wiki, the state file, the decision record and the log, ready for the next session.

In one sentence: the graph draws the map, memory remembers where you are, Jev picks the road at each junction, Claude does the thinking, and the loop makes sure nothing leaves until it has been checked.

Two warnings the analogies can hide.

- **A loop is only as good as its checker.** Re-reading your own work with the same assumptions misses the same mistakes, which is why the vault's checker gets a fresh context.

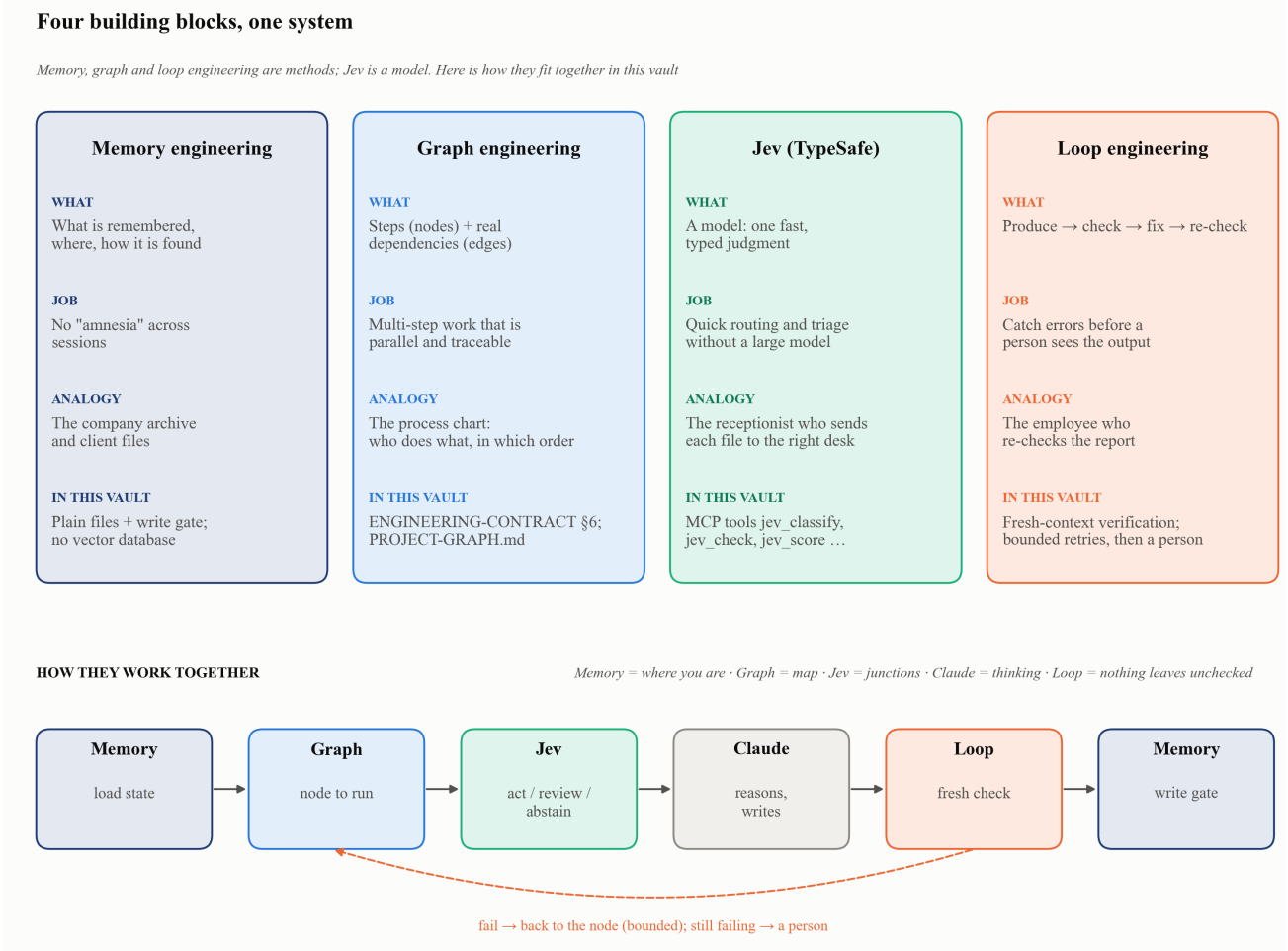


Figure 13: Memory engineering, graph engineering, Jev and loop engineering, and how they connect

- **A loop needs a stopping rule.** Without a retry limit and an escalation path, "fix it until it's right" can run forever or quietly lower the standard.

---

4. Worked Example: Follow One Source Through the System

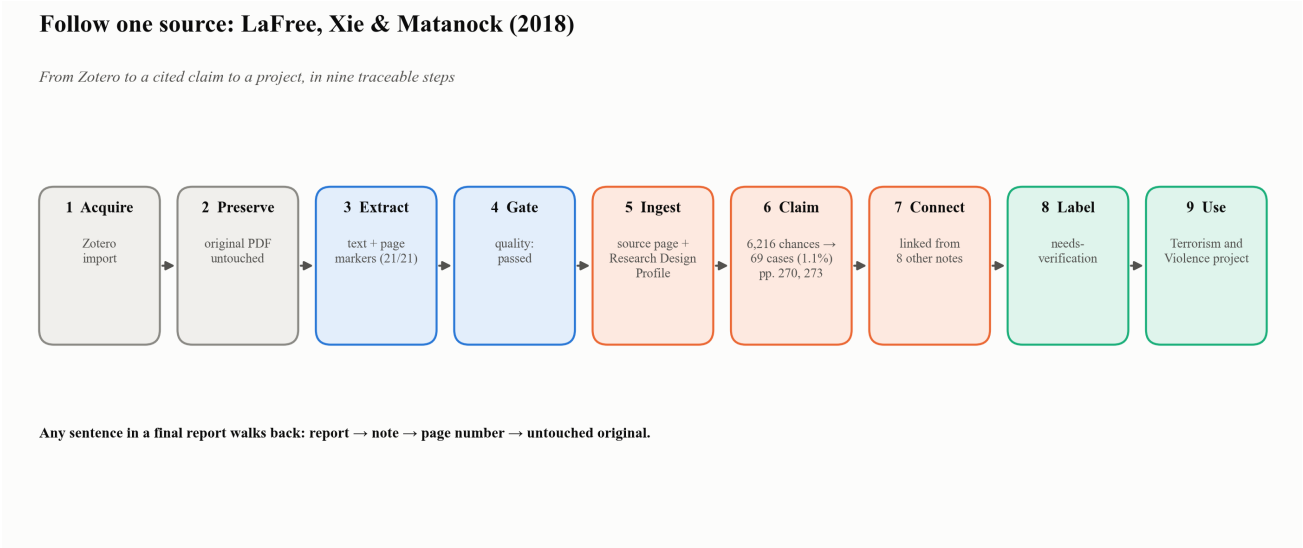


Figure 14: One source, end to end

The source: LaFree, Xie and Matanock (2018), "The Contagious Diffusion of Worldwide Terrorism: Is It Less Common Than We Might Think?", *Studies in Conflict & Terrorism*.

Step	What happened	Where you can see it
1. Acquire	Pulled from the Zotero library during the 2026-09-02 collection-by-collection ingestion	`01-Library/Reports/…__zotero.md`
2. Preserve	The PDF is stored unchanged	`01-Library/Originals/LaFree - 2018 - …pdf`
3. Extract	Native text extracted with `--- PDF PAGE n ---` markers (21 of 21 pages have text)	`01-Library/Text/…__text.txt`
4. Quality gate	`quality_status: passed`	`01-Library/Reports/…__quality.md`
5. Ingest	A source page (with Research Design Profile) plus atomic claim notes	`02-Wiki/lafree-2018-contagious-diffusion-of-terrorism.md` and others
6. One atomic claim	"Cross-national diffusion of terrorism is rare": of 6,216 opportunities for contagious diffusion (1970–2013), 69 occurred (1.1%), **with page citations** (pp. 270, 273)	`02-Wiki/cross-national-diffusion-of-terrorism-is-rare.md`

Step	What happened	Where you can see it
7. Connect	Linked to diffusion types, LISA spatial statistics, geographic concentration since 2003; referenced by 8 other notes	Related Notes section; Obsidian graph
8. Label honestly	`status: needs-verification` until a person checks the page numbers against the PDF	YAML header
9. Use in a project	The note is part of the conflict-studies cluster that served as the interpretive backbone of the Terrorism and Violence project	Project state

The point. Any sentence in a final report can be walked back to a note, to a page number, to an untouched original, in a few clicks.

---

5. One Core, Many Projects

Every project gets the same three files. Everything else depends on the project.

File	Purpose
`PROJECT-STATE.md`	Objective, phase, blockers, next actions: the single current truth
`DECISIONS.md`	Append-only record of consequential choices, with reasons and revisit triggers
`PROJECT-GRAPH.md`	The workflow graph: nodes, real dependencies, completion gates

A standard ten-point prompt creates a project: create the folder → state → decisions → graph if complex → find related notes through the index → list missing information and assumptions → show the structure → write → update state after each session → record only real decisions → never restructure personal notes without permission.

Case A: Data analysis (Terrorism and Violence)

- **Question:** How has terrorism evolved since 9/11, and what are the risks for Europe and the US?
- **What the system did:** Documented data provenance (ACLED, GRID, UCDP, Europol TE-SAT) with coverage caveats. Ran descriptive analyses as parallel graph nodes. Compared seven forecasting models in one rolling-origin backtest (Theta won *on this data*; the tool re-runs per series). Ran a fresh-context audit of published numbers. Delivered a report, Excel workbook, interactive HTML and **three audience versions of one deck**.
- **What the second brain added:** a terrorism/conflict-studies literature cluster (94 indexed notes, ingested from Zotero) linked to the data. The forecasting tool became shared, vault-wide infrastructure.

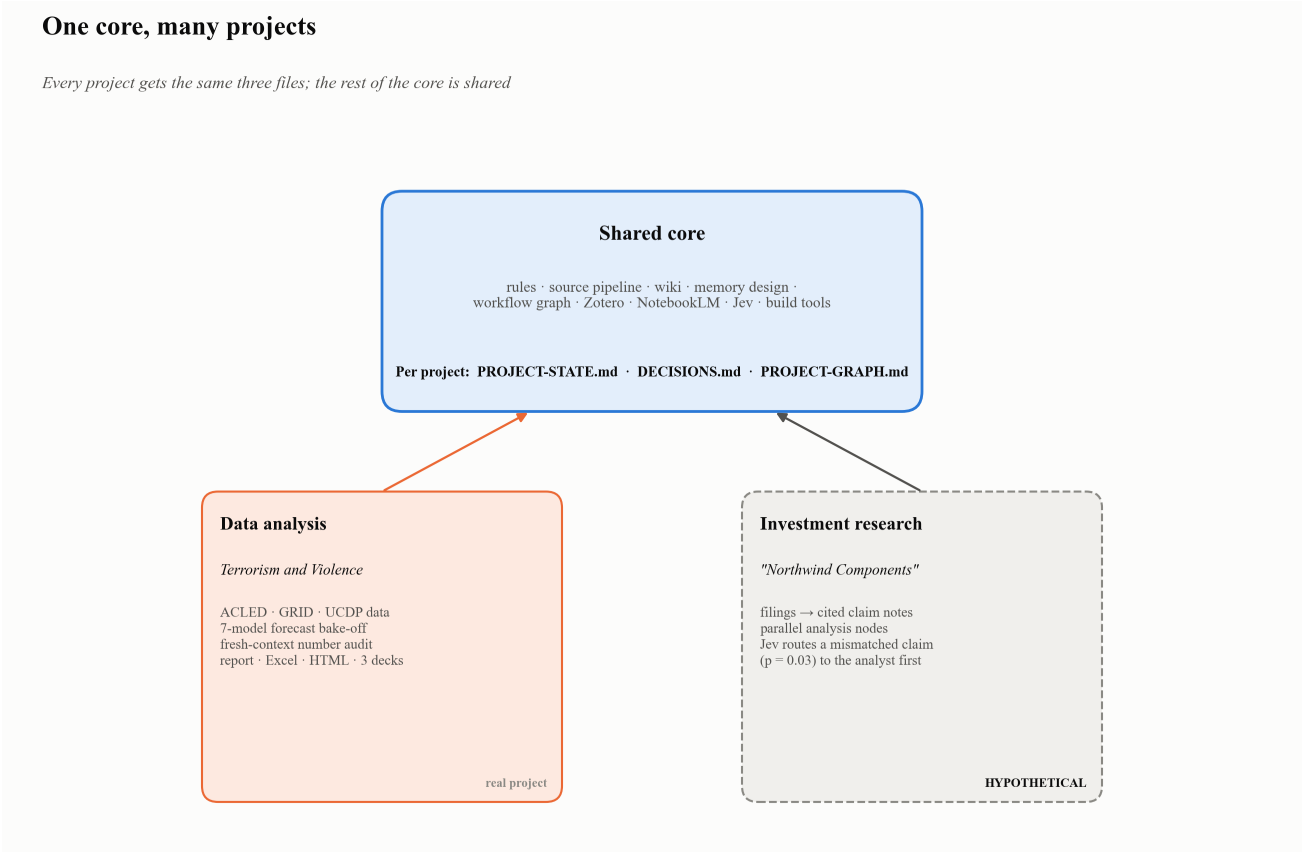


Figure 15: Same core, different projects

Case B: Investment research (hypothetical, illustrative only)

This example is invented to show how the method transfers. "Northwind Components" is a fictional company. The Jev results are real tool outputs on invented inputs (run 2026-09-28).

- **Setup:** Filings, earnings-call transcripts and analyst notes go to `01–Library/`. Claims ("segment X grew 18%") become atomic notes with page citations. The investment memo is a project folder.
- **Graph:** separate parallel nodes for financials, competitors, management guidance and risks → barrier → fresh-context check of every number against the filing → memo.
- **Jev as a triage step, not a verifier.** Before the analyst's check, Jev screens each draft claim against the passage it cites and routes it:

Draft claim	Jev screening result	Routing	Who decides
"*Total* revenue grew 18%"; the cited passage says total grew 12% and the *data-center segment* grew 18%	Supported by the passage? probability **0.03** → **no**	Flagged, sent to the front of the review queue	The analyst checks the filing and corrects the claim
"Data-center segment revenue grew 18%; industrial shrank"	Supported by the passage? probability **0.90** → **yes**	Lower review priority	The analyst still confirms it against the filing
"Management says the new plant will double capacity by 2027"	Source type: **management guidance** (confidence 1.00)	Tagged as forward-looking	Treated as unverified intent, never as fact

- **Value:** the analyst's attention goes first to the claims most likely to be wrong, and forward-looking statements stay labelled. Verification itself stays with the analyst and the source document. Jev does no arithmetic here: it judges whether a passage supports a claim's wording, and the numbers are compared by the analyst or by code.

Other fits (by design, not yet run as projects): policy monitoring with freshness dates, literature reviews, due diligence, grant proposals, internal knowledge bases.

---

6. Before and After

	General AI chat	The Second Brain
Memory	Lost between chats	State, decisions, knowledge and preferences each in a defined, inspectable place
Sources	Often unnamed or invented	Original preserved; page-level citation; quality gate
Uncertainty	Hidden behind fluent prose	Explicit labels: needs-verification, evidence strength, freshness, abstain
Large tasks	One long conversation	Workflow graph with parallel nodes and independent verification
Reuse	Start over each time	Knowledge base and tools compound across projects

	General AI chat	The Second Brain
Audit	Not possible	Git history, `log.md`, `DECISIONS.md`, linter, automated acceptance tests

---

7. Honest Limits

- **Verification is still human work.** Some notes and sources will need a person to check them against the original. The system labels them, so that work is visible rather than hidden.
- **Structure has a cost.** Setup, intake and verification take longer than asking a chatbot. The payoff comes with reuse and with stakes.
- **Integrations are uneven.** Zotero is programmatic; NotebookLM works only through the web interface (subject to quotas); several connectors need the owner's sign-in.
- **Jev is a decision aid, not evidence.** It scores the question it is given against the text it is given. A badly posed question gets a confident answer to the wrong question. Content sent to Jev leaves the machine for the TypeSafe service, so confidential, student-identifiable or regulated data must not be sent.
- **Rules are guidance to the AI, not physical enforcement.** They work because they are loaded every session and checked by tests and review. People still approve every consequential write.
- **Benefits are design intentions.** No controlled study of time saved or error reduction has been run in this vault.

---

8. Glossary

Term	Meaning
Atomic note	A wiki page holding one primary idea with its sources
Barrier	A point in a workflow that waits until all required parallel steps finish
Claude Code	Anthropic's AI assistant that works directly on files and tools on your computer
Fan-out	Splitting work into independent steps that can run in parallel
Fresh-context verification	Checking results with a reviewer that sees sources and outputs but not the worker's reasoning
Freshness class	durable / periodic / dynamic: how quickly a claim can go out of date
Jev	TypeSafe's model for bounded, calibrated judgments, used in Claude Code through a community MCP plugin with six tools
MCP	Model Context Protocol: the standard way tools such as Jev plug into Claude
Scope (plugin)	Where a Claude Code plugin is installed: user (you, every project), project (shared through the repository) or local (you, this repository only)

Term	Meaning
Provenance	The traceable origin of a claim: source, page, processing history
Skill	A packaged, reusable procedure Claude loads when a task calls for it
Vault	The Obsidian folder that holds the whole knowledge base
Write gate	The checklist that must pass before anything is saved

Appendix A. Build Your Own: A Tell–Show–Do Setup Guide

Each step has three parts. **Tell** says why the step exists. **Show** gives the file, command or setting. **Do** is a check you can run to confirm the step worked. Follow the steps in order; each builds on the previous one.

> **Security rule for every step.** Never paste an API key into a Claude conversation, a screenshot, a shared document, a Git repository or the text of a command. Enter keys at a hidden Terminal prompt (`read -s`) so they are never shown or recorded, and keep key files out of version control. Rotate any key that has been exposed.

Prerequisites

Tool	Purpose
Claude Code	The AI assistant that works on your files
Obsidian	Viewer and editor for the vault
Git	Version history for the whole vault
Python 3.11+ with PyYAML; poppler (`pdftotext`, `pdfinfo`), `ocrmypdf`, Tesseract language packs	Source pipeline
pandoc (plus a LaTeX engine for PDF)	Word and PDF deliverables
Zotero desktop (optional)	Reference library integration
Node.js 20.12 or newer	Required by the Jev plugin, which launches through `npx` (check with `node --version`)
A TypeSafe account (for Jev)	Issues the API key

Step 1: Create the vault skeleton

- **Tell.** A fixed folder structure means that both people and the AI always know where things belong.
- **Show.** Create these folders in one directory and open it in Obsidian as a vault: `00-System/`, `01-Library/{Originals,OCR,Text,Reports,Processed}/`, `02-Wiki/`, `03-Projects/`, `04-Templates/`, `05-Inbox/`. Add `index.md` and `log.md`, then run `git init`.
- **Do.** Obsidian opens the folder, and `git status` shows the new files.

Step 2: Write the rulebook

- **Tell.** Behaviour is set by files that anyone can read, not by hidden prompts.
- **Show.**
  - `RESEARCH_PROTOCOL.md`: no invented citations, quotations or metadata; separate source claims from interpretation; state uncertainty.

  - `CLAUDE.md`: workflows (`ingest`, `query vault`, `save`, `lint wiki`, `weekly review`), the page template with YAML frontmatter (`status: needs-verification` by default), file naming, change-safety rules.
- **Do.** Start a Claude Code session in the vault and ask: "Which file wins if CLAUDE.md and RESEARCH_PROTOCOL.md conflict?" The correct answer is `RESEARCH_PROTOCOL.md`.

Step 3: Add memory and graph engineering

- **Tell.** This step separates kinds of memory and turns large tasks into graphs with real dependencies.
- **Show.**
  - `00-System/START-HERE.md`: task router and context budget (at most three system documents at start; wiki notes in batches of five).
  - `00-System/ENGINEERING-CONTRACT.md`: memory types and write gate; provenance labels; freshness classes; index-first retrieval; workflow-graph node contract; link classification.
  - Templates for `PROJECT-STATE.md` and `DECISIONS.md` in `04-Templates/`.
- **Do.** Add a read-only linter and an acceptance test script (this vault uses `lint-vault.py` and `test-vault-system.py`). Run them; they must pass without modifying any file.

Step 4: Install the source pipeline

- **Tell.** The AI should never read a scanned PDF blind. Text is extracted once, page-marked and quality-checked.
- **Show.** Install the pipeline dependencies, then run the pipeline on a PDF:

```
python3 00-System/Scripts/inspect-pdf.py --file "05-Inbox/new-source.pdf"
python3 00-System/Scripts/process-source.py --file "01-Library/Originals/Author - Year - Title.pd
cat "01-Library/Reports/Author - Year - Title__quality.md"
```

- **Do.** The text file contains `--- PDF PAGE n ---` markers, and the report shows `quality_status: passed` or `usable-with-review`.

Step 5: Connect Zotero

- **Tell.** Bibliographic metadata comes from your library, not from the AI's memory.
- **Show.**
  - In Zotero: **Zotero → Preferences → Advanced → General** (**Edit → Preferences** on Windows) → check **"Allow other applications on this computer to communicate with Zotero."** This enables read-only local access.
  - For write-back (Research Design Profiles as Zotero notes): create a Web API key with write access at zotero.org and save it with `!` to a git-ignored file (this vault uses `00-System/Config/.zotero-web-api-key`).
- **Do.** `/zotero-sources doctor` reports that the API is reachable; `/zotero-sources collections` lists your collections.

Step 6: Add Jev (calibrated judgments)

Source of truth: the audited Jev installation deck (17 slides, September 2026). Supplementary details come from its companion guide and are marked as such.

- **Tell.** Some repeated decisions should come with a probability and the option to abstain. You are installing three things: **Claude Code** (where you type), the **Jev MCP plugin** (six decision tools plus an agent skill,

community-maintained rather than an official TypeSafe or Anthropic product) and a **TypeSafe API key** (without it, calls fail with `no_api_key` or `401`). The separate TypeSafe workflow skill is optional and is not required for Jev.

- **Know the two interfaces.** The *normal Terminal* prompt ends with `%` or `$`: shell commands, no slash. *Inside Claude Code*, the prompt starts with the Claude chevron: slash commands, typed one at a time, with Esc to close menus.

Show and do, step by step:

1. **Create a TypeSafe API key** (browser). Go to `https://console.typesafe.ai/settings/keys`, sign in, create a key with a clear name (e.g., "Claude Code – MacBook"), and copy it once into a password manager. Use a direct TypeSafe key; keys from other AI gateways will not authenticate (companion guide).

2. **Add the marketplace** (inside Claude Code):

   ```
   /plugin marketplace add rashedInt32/jev-mcp
   ```

   If an "Add Marketplace" box opens, enter only `rashedInt32/jev-mcp`; never paste the install command into that box. *Check:* `Successfully added marketplace: jev-mcp`.

3. **Install the plugin** (inside Claude Code):

   ```
   /plugin install jev@jev-mcp
   /reload-plugins
   ```

   When asked, choose **Install for you (user scope)**. No separate skill installation is needed. *Check:* Jev appears once, at user scope.

4. **Save the key in a protected file** (normal Terminal, one line at a time):

   ```
   mkdir -p ~/.config/typesafe
   read -s "KEY?Paste your TypeSafe API key, then press Enter: "
   printf '%s' "$KEY" > ~/.config/typesafe/key
   chmod 600 ~/.config/typesafe/key
   unset KEY
   ```

   Paste the real key at the hidden prompt; nothing appears on screen. Never type a placeholder such as `YOUR_KEY`, which produces `401`. The plugin looks for `TYPESAFE_API_KEY`, then `JEV_API_KEY`, then this file; the file is the most reliable option (companion guide).

5. **Check the installation** (normal Terminal): `claude plugin list`. *Check:* `jev@jev-mcp` appears **once**, scope **user**, status **enabled**. A separate TypeSafe skill entry is optional, and your version numbers may be newer.

6. **Confirm the MCP connection** (inside Claude Code): `/mcp`. *Check:* `plugin:jev:jev` with a green check and **six tools**. Press Esc to close. A green check shows the connection, not that the key works.

7. **Test the key** (inside Claude Code): "Call the Jev jev_models tool and show me the available models." *Check:* models are listed (`jev-latest`, recommended; `jev-preview`, whose behaviour may change). A `401` means the saved key is missing, invalid or expired: replace it, restart Claude Code and test again.
8. **Run a first real decision** (inside Claude Code): "Use Jev to classify: 'My card was charged twice and I want a refund.' Choices: billing, technical, sales, other. Show all probabilities and confidence." *Check:* billing 1.00, confidence 1.00, recommended action **act**.

Fix a duplicate installation. If `claude plugin list` shows Jev twice (project scope and user scope), an older project copy can override the newer user copy inside that repository. Remove the project copy **from inside that**

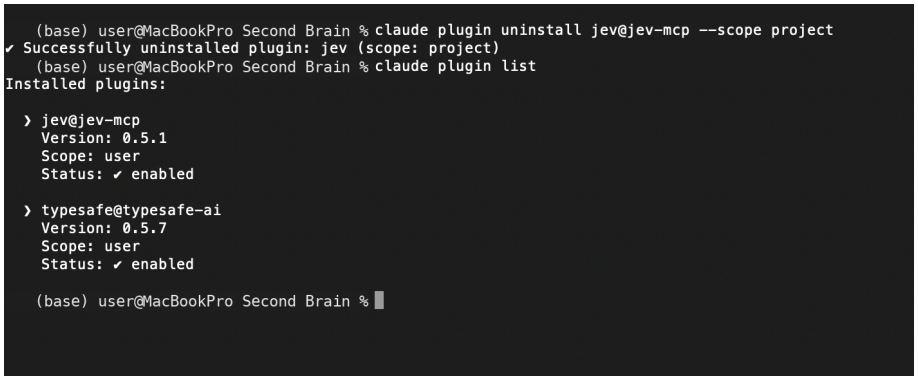


Figure 16: A clean installation: Jev once at user scope. The TypeSafe skill entry is optional. Screenshot from the audited deck

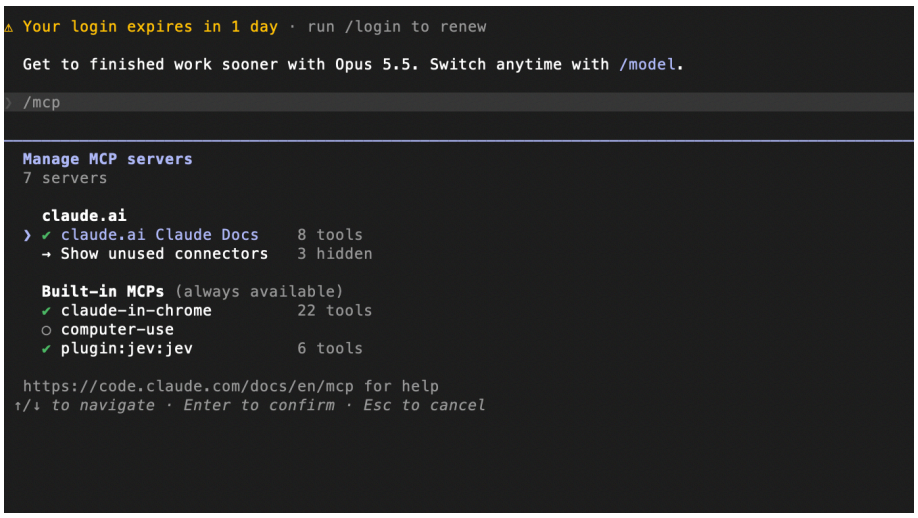


Figure 17: The Jev MCP server connected with six tools. Screenshot from the audited deck

project folder; run from your home folder, the command may only find the user installation:

```
cd "/path/to/the/project/where-Jev-is-installed"
claude plugin uninstall jev@jev-mcp --scope project
claude plugin list
```

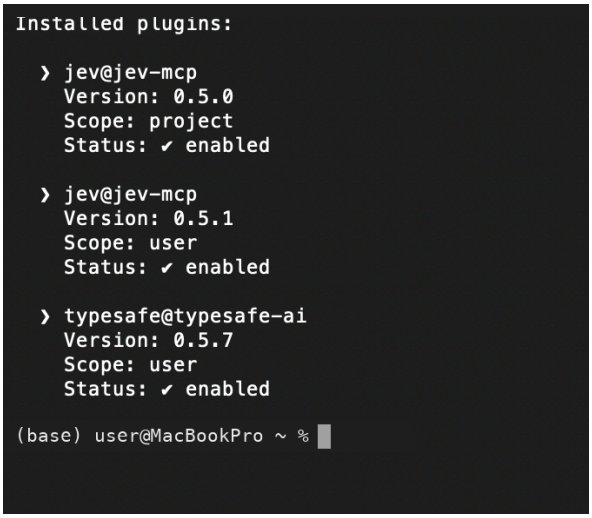


Figure 18: The duplicate-installation symptom: Jev at both project and user scope. Screenshot from the audited deck

Troubleshooting by symptom:

Symptom	Fix
`no_api_key`	Create `~/.config/typesafe/key`, set permission 600, restart Claude Code
`401` authentication	Generate a new TypeSafe key and replace the file contents
Red × in `/mcp`	Restart Claude Code; if it persists, run `claude --debug` in the normal Terminal (check Node.js ≥ 20.12; the first launch can take a few seconds while `npx` downloads the package)
"Marketplace source invalid"	Run `/plugin marketplace add`, then enter only `rashedInt32/jev-mcp`
Project uninstall targets user	`cd` into the affected project before using `--scope project`
Plugin update installed	`/reload-plugins`; restart Claude Code only if the old version stays loaded

Optional: let Claude use Jev automatically. Add this rule to the project's `CLAUDE.md`:

> Use Jev automatically when the task applies the same bounded judgment to many items, the allowed outputs are explicit, and confidence changes the next action. Use ordinary reasoning for one-off discussion. Do not ask Jev to perform arithmetic or date comparison. Never present Jev output as research evidence, factual verification, citation verification, or source validation.

Good uses: classifying many notes or records; ranking passages after ordinary retrieval; routing ambiguous cases

for review. **Do not send** student-identifiable data, unpublished confidential research, or secrets, credentials and regulated data. Use synthetic, redacted or de-identified text wherever possible.

Final checklist: key created · marketplace added · plugin installed once at user scope · key saved in `~/.config/typesafe/key` (mode 600) · `plugin:jev:jev` green with six tools · `jev_models` succeeds · `jev_classify` returns probabilities and confidence. Then describe the real task, ask Claude to use Jev, and **review uncertain cases instead of automating them blindly**.

Step 7: Use NotebookLM (optional)

- **Tell.** NotebookLM produces formats Claude does not produce itself: mind maps, infographics, audio overviews.
- **Show.** There is no API. Install the **Claude in Chrome** extension, sign in to NotebookLM in Chrome, and ask Claude to add a vault document as a source and generate the output.
- **Do.** Treat outputs as derived material. Regenerate them whenever the underlying facts are corrected, and expect usage quotas.

Step 8: Start your first project

- **Tell.** Every project gets the same state, decision and graph files, so any future session can pick up where the last one stopped.
- **Show.** Paste this form into Claude Code, with your own values in place of the [brackets]. Leave a field blank if you don't know it yet.

```
Create a new project in the Second Brain.

Project name: [short name, e.g. Drivers of Civil War]
Objective: [one or two sentences: the question this project answers]
Audience: [who will read the outputs, e.g. students, executives, reviewers]
Final deliverables: [e.g. Word report, infographic, diagrams, slide deck]
Available sources: [vault folders, Zotero collections or files, or "search the vault"]
Scope and limits: [what is in and out: period, regions, source types]
Success criteria: [how the result will be judged, e.g. "I will review the outputs"]
Deadline or review date: [YYYY-MM-DD]
Language: [e.g. English]
```

What happens when you send it. You don't type any instructions after the form. Claude recognises it and follows the *NEW PROJECT Workflow* written in `00-System/VAULT-WORKFLOWS.md`. `CLAUDE.md`, which is loaded at the start of every session, points it there. The workflow has eleven steps:

1. **It plans first and waits.** It briefly shows the folders and files it intends to create, then stops. Nothing is written until you approve.
2. **It creates the project folder** in `03-Projects/`, with a short English name.
3. **It writes `PROJECT-STATE.md`.** This is the project's one-page memory: the objective, current state, open items, last decision and why, a "Tried and rejected — don't retry" list, and a session log. Your form's fields (audience, deliverables, sources, scope, success criteria, deadline, language) are copied into it.
4. **It creates `DECISIONS.md`** for genuine, consequential decisions only, each with its reason. It stays empty rather than being padded.
5. **It creates `PROJECT-GRAPH.md` only if the work has several dependent steps.** The graph maps real dependencies only, so independent steps can later run side by side and be checked before they are

combined.

6. **It adds the project to the Project Router** in `00-System/START-HERE.md`, so future sessions can find it.
7. **It looks for related notes through `index.md`**, in small batches, without loading the whole vault.
8. **It lists what is missing and what it assumed.** Blank or vague fields and sources it could not find appear here. A blank field never blocks the start; the assumption is written down so you can correct it.
9. **It follows `RESEARCH_PROTOCOL.md`:** no invented sources, citations or quotations. Unverified claims are marked as such.
10. **It keeps the record at the end of every work session.** It updates `PROJECT-STATE.md` and adds a session-log entry: what was done, what was decided and why, what is open, and which approaches were tried and rejected.
11. **It never deletes, merges, renames or restructures your existing notes** without your permission.

When it finishes, it reports the files it created, the related notes it found, the missing information and assumptions, and a suggested first step. It also names the skill that fits your deliverables: a slide deck, a course page or an explainer video. The short command `new project: [name]` starts the same workflow; Claude then asks for the fields it needs.

- **Do.** Close the session, open a new one and ask "Where are we on this project?" A good answer comes from `PROJECT-STATE.md`, not from memory of the chat.

Step 9: Keep it healthy

- **Tell.** A knowledge base decays without maintenance.
- **Show.** Weekly: `weekly review` and `lint wiki` (both read-only). Monthly: re-run the acceptance tests and review freshness dates. Record preferences, not facts, in auto memory.
- **Do.** The lint report lists orphans, broken links and unsupported claims. You decide what to fix; nothing is merged or deleted automatically.

Appendix B. Sources and Provenance for This Guide

- **System files:** `CLAUDE.md`, `RESEARCH_PROTOCOL.md`, `00-System/START-HERE.md`, `00-System/ENGINEERING-CONTRACT.md`, `00-System/Source-Pipeline.md`, `00-System/Zotero/README.md`, `00-System/ACCEPTANCE-RESULTS.md`.
- **Activity records:** `log.md` (including the 2026-08-13 engineering build, Steps 5–12), Git history, project state files for the two named projects.
- **Jev (source of truth):** *Installing Jev for Claude Code*, audited presentation v2 (September 2026; 17 slides), with its companion guide for supplementary details. Upstream references cited in that deck: github.com/typesafe-ai/skills, github.com/rashedInt32/jev-mcp, code.claude.com/docs/en/plugins, code.claude.com/docs/en/mcp, console.typesafe.ai/settings/keys, typesafe.ai.
- **Live tool results:** the Jev outputs in Sections 3.6 and 5, run on 2026-09-28 on invented inputs, plus an earlier session result.
- **Explanation methods (external, general knowledge):** Minto's Pyramid Principle (answer first); Paivio's dual coding and Mayer's multimedia principles (one diagram per idea, labels on the diagram); Sweller's worked-example effect (follow one source); the C4 model's zoom levels (context before components); Tell–Show–Do instruction (this appendix).
- **Verification state:** claims in the guide were re-checked against vault files after drafting; three wording corrections were made (see the project's verification log).

License and How to Cite

© 2026 Isa Haskologlu, PhD. This work is licensed under the **Creative Commons Attribution 4.0 International License (CC BY 4.0)**: https://creativecommons.org/licenses/by/4.0/. You may share and adapt it for any purpose, provided you give appropriate credit, link to the license, and indicate if changes were made.

Suggested citation (APA 7): Haskologlu, I. (2026). *The Second Brain: An AI research system you can audit* [Guide and interactive hub]. https://claude.ai/artifact/EMh3hwYGRD6KdTSQFMEPVh